

National Textile University, Faisalabad



Department of Computer Science

Name:	Husnain Nadeem
Class:	BSCS '5A'
Registration No:	23-NTU-CS-1038
Lab Report:	Home task Aftermid
Course Name:	Embedded IOT FALL -2025
Submitted To:	Sir Nasir
Submission Date:	17 December 2025

Question-1 ESP32 Webserver (webserver.cpp)

Part A: Short Questions

1. What is the purpose of `WebServer server(80);` and what does port 80 represent?

This line creates an **object** (an instance) of the `WebServer` class named `server`.

- **The Number 80:** This represents the **TCP port number**.
- **Significance:** Port 80 is the standard port for **HTTP** (Hypertext Transfer Protocol). When you type an IP address into a web browser, it automatically tries to connect via port 80. By setting this, you are telling the ESP32 to listen for incoming web requests on that specific "channel."

2. Explain the role of `server.on("/", handleRoot);` in this program.

This is a path handler or a "routing" instruction.

- `"/`: This represents the "Root" or home directory of your website (the main page).
- `handleRoot`: This is the name of the function you want the ESP32 to execute when someone visits that path.
- In short: It tells the ESP32: "If a user navigates to the main IP address of this device, immediately run the code found inside the `handleRoot` function."

3. Why is `server.handleClient();` placed inside the `loop()` function? What will happen if it is removed?

The ESP32 is not "always" aware of a user trying to connect; it has to actively check.

- **The Role:** This function tells the ESP32 to check if any web browser has sent a request.
- **If removed:** The ESP32 will continue running its other tasks, but it will **never respond** to your web browser. Even if the ESP32 is connected to Wi-Fi, the website will never load because the "request" from your computer is being ignored by the processor.

4. In `handleRoot()`, explain the statement:

`server.send(200, "text/html", html);` This statement is the actual "reply" sent back to the user's browser. It consists of three parts:

- **200:** This is the **HTTP Status Code** for "OK." It tells the browser that the request was successful.
- **"text/html":** This is the **Content Type**. It tells the browser that the data being sent should be interpreted as a webpage (HTML), not just plain text or an image.
- **html:** This is the **Variable** containing the actual code (HTML/CSS) that creates the visual layout of your page.

5. What is the difference between displaying last measured sensor values and taking a fresh DHT reading inside `handleRoot()`?

This is a choice between **real-time accuracy** and **processing speed**:

- **Taking a fresh reading:** Every time you refresh the webpage, the ESP32 triggers the DHT sensor to take a new measurement. This provides the most current data but can cause a slight delay (latency) because DHT sensors are relatively slow to read.
- **Displaying last measured values:** The ESP32 reads the sensor in the background (perhaps every 2 seconds) and stores it in a variable. When you refresh the page, it simply shows that stored number. This makes the website feel much **faster and more responsive**, though the data might be a few seconds old.

Part B: Long Question

Working of ESP32 Webserver-Based Temperature and Humidity Monitoring System

This system requires an ESP32, DHT22 sensor, OLED display, push button, and a web server. ESP32 is the microcontroller that is responsible for processing of circuit, DHT22 sensor for reading temperature and humidity, OLED display for showing current temperature and humidity values, push button whenever pressed gives latest values for temperature and humidity.

1. Wi-Fi Connection and IP Assignment

The process begins with the ESP32 initializing its Wi-Fi radio in Station (STA) Mode.

The Process: Using the `WiFi.begin(ssid, password)` command, the ESP32 attempts to handshake with the local wireless router.

IP Assignment: Once the router authenticates the ESP32, it assigns it a local IP address (e.g., 192.168.1.15) via DHCP.

User Interface: To make the system user-friendly, the ESP32 is programmed to display this specific IP address on the OLED screen. This allows the user to know exactly what address to type into their smartphone or computer browser to access the monitoring dashboard.

2:Web Server Initialization and Request Handling

After the Wi-Fi connection is stable, the Web Server is started on Port 80.

The Server Object: An object (typically server) is created to manage incoming traffic.

Routing: The command `server.on("/", handleRoot);` sets up a "listener" for the home page.

Request Handling: The `server.handleClient()` function is placed in the `loop()`. This acts like a receptionist who constantly checks for visitors. When a user enters the IP address in a browser, `handleClient()` detects the request and triggers the `handleRoot()` function to deliver the webpage data.

3:Button-Based Sensor Reading and OLED Update

Unlike systems that read sensors on a timer, this specific design uses a Push Button for manual updates.

The Mechanism: The button is configured with `INPUT_PULLUP`, meaning the pin stays "High" until the button is pressed (completing the circuit to "Low").

Triggering a Read: When the ESP32 detects this state change in the `loop()`, it calls the `readDHTValues()` function.

Output: The DHT22 sensor measures the ambient air. These new values are immediately saved into variables and pushed to the OLED display, ensuring that the physical device always shows the most current data as soon as the button is pressed.

4:Dynamic HTML Webpage Generation

The webpage is not a static file; it is a dynamically generated string of HTML code.

Variable Injection: Inside the `handleRoot()` function, the current temperature and humidity variables are embedded directly into the HTML string.

Dynamic Content: If the sensor hasn't been read yet, the code can use "if" statements to display a placeholder message (e.g., "Press button for data") instead of blank values. When the user refreshes the page, the string is re-built with whatever values are currently stored in the memory.

5:Purpose of the Meta Refresh Tag

The meta refresh tag is a crucial piece of HTML code: `<meta http-equiv='refresh' content='5'>`.

The Logic: This tells the web browser to automatically reload the page every 5 seconds.

The Benefit: Without this, the user would have to manually click the "Reload" button on their browser to see updated data. It bridges the gap between the physical button press and the digital display, ensuring the web dashboard stays synchronized with the hardware.

6: - Common issues in ESP32 webserver projects and their solutions

Some **common issues** in ESP32 webserver projects include Wi-Fi connection delay, invalid DHT sensor readings, and webpage not updating. These issues can be solved by checking the Wi-Fi credentials, double checking the sensor pins, adding small delays, and by placing the server handling code inside the loop.

Question-2 Blynk Cloud Interfacing (blynk.cpp)

Part-A: Short Questions

1. What is the role of Blynk Template ID in an ESP32 IoT project? Why must it match the cloud template?

The Blynk Template ID is a unique code that tells the Blynk Cloud which project template the ESP32 belongs to. It links the device to the correct dashboard layout created on the Blynk platform. This ID must match the cloud template because the cloud uses it to identify where the incoming data should appear. If the Template ID is incorrect, the ESP32 will either fail to connect or the data will not show on the dashboard properly.

2. Differentiate between Blynk Template ID and Blynk Auth Token.

The Blynk Template ID identifies the project or template on the Blynk Cloud and remains the same for all devices using that project. In contrast, the Blynk Auth Token is a unique authentication key assigned to each individual device. The Template ID links the device to a project, while the Auth Token verifies and authorizes that specific device to communicate with the cloud.

3. Why does using DHT22 code with a DHT11 sensor produce incorrect readings?

Mention one key difference between the two sensors.

DHT22 and DHT11 have different ranges for measuring temperature and humidity. **DHT22** can measure temperature from -40 °C to 80 °C (with $\pm 0.5^{\circ}\text{C}$ accuracy) and humidity from 0% to 100% RH (with $\pm 2\%$ RH accuracy) while **DHT11** can measure temperature from 0 °C to 50 °C (with 2°C accuracy) and humidity from 20% to 90% RH (with $\pm 5\%$ RH accuracy)

Reason for incorrect readings

Using DHT22 code with DHT11 causes wrong values because the sensor type does not match and the code is trying to read data that isn't there. DHT11 only provides whole numbers (like 25 degrees), while DHT22 provides floating numbers (like 25.5 degrees). The code for DHT22 expects those extra decimal bits and when it doesn't find them in the DHT11's data stream, it provides incorrect readings.

4. What are Virtual Pins in Blynk? Why are they preferred over physical GPIO pins for cloud communication?

Virtual Pins in Blynk are software-based communication channels that allow data exchange between the ESP32, the Blynk Cloud, and the mobile or web dashboard. They are not connected to any physical pins on the microcontroller. Virtual Pins are preferred because they offer flexibility, better organization, and easier cloud communication without depending on actual hardware GPIO pins.

5. What is the purpose of using BlynkTimer instead of delay() in ESP32 IoT applications?

BlynkTimer is used to perform tasks at regular intervals without pausing the entire program. Unlike delay(), which blocks execution and disrupts Wi-Fi and Blynk communication, BlynkTimer works in a non-blocking way. ESP32 hardware timers use interrupts, where calling Wi-Fi, Blynk, or display functions is unsafe. BlynkTimer avoids these issues and keeps the system responsive and stable.

Part-B: Long Question

Workflow of Interfacing ESP32 with Blynk Cloud

This project makes use of an ESP32 microcontroller along with a DHT22 sensor, an OLED display, a push button, and the Blynk Cloud platform to monitor and display temperature and humidity values.

- **Creation of Blynk Template and Datastreams** First, a template is created on the Blynk Cloud. Inside this template, separate datastreams are defined for temperature and humidity. In the program, virtual pin V0 is assigned for temperature and virtual pin V1 is assigned for humidity. These datastreams help organize the data sent from the ESP32 and allow it to be shown properly on the Blynk dashboard.
- **Role of Template ID, Template Name, and Auth Token** After creating the template, important credentials are generated, including **BLYNK_TEMPLATE_ID, BLYNK_TEMPLATE_NAME, and BLYNK_AUTH_TOKEN**. The Template ID uniquely identifies the project template on the Blynk Cloud and links the ESP32 to the correct dashboard. The Auth Token is a unique key given to each device so it can authenticate and communicate with the Blynk Cloud. The Template Name is simply the readable name of the project, such as “dht” in the given code, and is mainly used for identification.
- **Sensor configuration issues (DHT11 vs DHT22)** The DHT22 sensor is used to measure temperature and humidity in this project. If a DHT11 sensor is connected while the code is written for DHT22, the readings will be incorrect. This happens because the DHT11 only sends integer values, while the DHT22 provides decimal values with higher accuracy. Since the DHT22 code expects floating-point data, using a DHT11 sensor causes mismatched data and wrong readings. Therefore, the sensor type in the code must always match the actual hardware used.
- **Sending data using Blynk.virtualWrite()** The function `readAndDisplayAndSend()` is responsible for reading data from the DHT22 sensor. It updates the OLED display with the latest temperature and humidity values and then sends this data to the Blynk Cloud using `Blynk.virtualWrite(V0, t)` for temperature and `Blynk.virtualWrite(V1, h)` for humidity. A push button is also included to manually trigger new sensor readings, and pressing the button calls the same function. In addition, a `BlynkTimer` is used to automatically send updated data every five seconds without blocking the program's execution.

- Common problems faced during configuration and their solutions** Common issues in ESP32 and Blynk-based projects include failure to connect to the Blynk Cloud, incorrect sensor readings, or the dashboard not updating as expected. These problems can usually be fixed by checking Wi-Fi credentials, ensuring the Template ID and Auth Token match the cloud configuration, verifying that the correct DHT sensor type is selected in the code, and making sure `Blynk.run()` and `timer.run()` are placed inside the loop function.

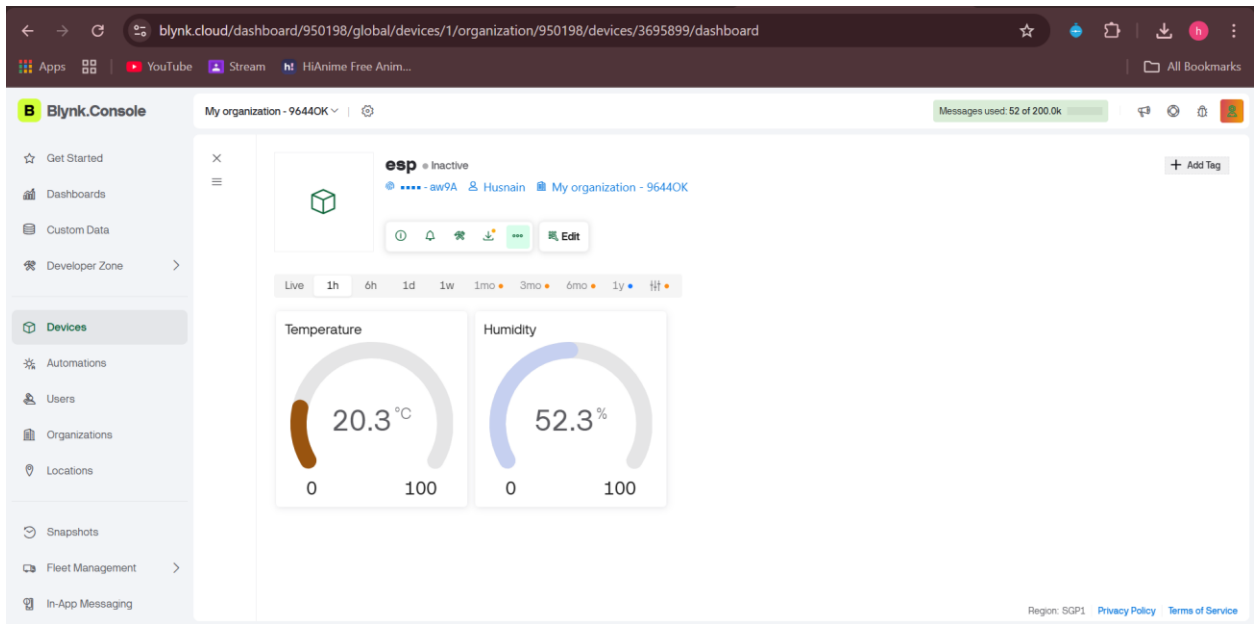
WEB DASHBOARD

- For the web dashboard setup, the user logs into Blynk using an email account and goes to the developer zone to create a new template with the required datastreams. After that, a device is added to the template. Adding a device generates the Template ID, Template Name, and Auth Token, which are then copied into the ESP32 code before uploading it to the board.

The screenshot shows the Blynk Console web interface. The left sidebar contains navigation options: Get Started, Dashboards, Custom Data, Developer Zone (selected), Devices, Automations, Users, Organizations, Locations, Snapshots, Fleet Management, and In-App Messaging. The main content area displays the 'Datastreams' configuration for a template named 'Dht by husnain'. A table lists the configured datastreams:

ID	Name	Pin	Color	Data Type	Units	Is Raw	Min
1	Temperature	V0	Orange	Double	°C	false	0
2	Humidity	V1	Blue	Double	%	false	0

At the bottom right, it indicates 'Region: SGP1' and provides links for 'Privacy Policy' and 'Terms of Service'.



Mobile Dashboard

